Soft Robot Mechanics Group | Institute of Applied Mechanics | Universität Stuttgart

Curling Spiral Simulation

Weekly work summary

Abdul Rehman
1-23-2026

Mentor: George Wenson

Table of Content

Week 1: Degrees of Freedom Analysis and Spiral Mechanism Design in SolidWorks 3

1. Theoretical Derivation of Degrees of Freedom 3

2. Parametric Decomposition via Triangulation 3

3. Design Specification and Dimensional Synthesis 3

4. CAD Implementation and Assembly 4

5. Spiral (Mathematical Foundation) 5

Week 2: Literature review and Simulation in Abaqus 6

6. Research paper SpiRobs summary: 6

7. Research paper Elephant-Inspired grasping summary: 6

8. Modeling and Simulation Strategy in Abaqus 7

1. Modeling Approach and Skill Acquisition 7
2. Geometry and Domain Discretization 7
3. Material Constitutive Models 9
4. Meshing and Element Formulation 9
5. Result Visualization 10

Week 3: Abaqus scripting with vector mathematics 11

9. Coordinate Transformation 11

1. Trigonometric Projection and Local-to-Global Mapping 12
2. Linear Intersection for Vertex Determination 12
3. Vector Normalization and Geometric Offsetting 12
4. Orthogonal Distance from a Point to a Line Segment 12

10. User-Defined Parametric Variables 13

1. Global Scaling and Base Geometry 13
2. Cut and Hinge Geometry 14
3. Material and Mesh Properties 14

11. Custom Scripting Functions 14

1. intersect(pt1, theta1, pt2, theta2) 15
2. dist_point_to_segment(px, py, P1, P2) 15
3. apply_partition(plane_feature) 15
4. build_face_seq(face_list) 16

Week 4: Data collection & Logarithmic spiral fitting 17

12. Interactive Logarithmic Spiral 17

13. Data fetching from Abaqus CAE 17

1. Starting coordinate Issue 17

2. Visualizing the Deformation 18

14. Mapping of the Spiral: Code Implementation and Curve Fitting 18

1. Automated Fitting Methodology 19
2. Parameter Trend Analysis 20
3. Diagnosis of Fitting Failure at high Angles (>151°) 20
4. Visualizing the Spiral Fit Across Actuation Angles 20

Week 1: Degrees of Freedom Analysis and Spiral Mechanism Design in SolidWorks

1. Theoretical Derivation of Degrees of Freedom

To define the geometry of a planar quadrilateral for the proposed mechanism, we must first establish the minimum number of independent parameters required to define its shape.

A planar quadrilateral consists of four vertices. In a two-dimensional Cartesian coordinate system, each vertex is defined by an (x, y) coordinate pair, resulting in an initial total of 8 degrees of freedom (DOF):

4 vertices x 2 coordinates/vertex = 8 DOF

However, for shape definition, we must eliminate rigid body motions (translation and rotation) which do not affect the intrinsic geometry:

- **Translation:** Fixing one vertex at the origin (0,0) constrains 2 DOF.
- **Rotation:** Aligning one adjacent side along a principal axis (e.g., the x-axis) constrains 1 rotational DOF.

Subtracting these rigid body motions leaves us with the minimum independent parameters required to define the shape:

8 DOF - 2 (translation) - 1 (Rotation) = 5 DOF

2. Parametric Decomposition via Triangulation

An alternative verification of this result can be achieved by decomposing the quadrilateral into two adjacent triangles.

- **Triangle 1:** A general triangle requires 3 parameters to be fully defined (e.g., Side-Side-Side, Side-Angle-Side, or Angle-Angle-Side).
- **Triangle 2:** The second triangle shares a common side with the first. Since this shared side is already defined by the first triangle, the second triangle requires only 2 additional parameters (e.g., Angle-Angle, Side-Side or Side-Angle) to be fixed relative to the first.

Total Independent Parameters: *Three (triangle 1) + two (triangle 2) = 5 DOF*

Consequently, a general quadrilateral can be fully defined by valid combinations of 5 parameters, such as:

- 4 Sides and 1 Angle
- 3 Sides and 2 Angles
- 2 Adjacent Sides and 3 Angles *(used in this simulation study)*

3. Design Specification and Dimensional Synthesis

For the specific application detailed in the *technical report SPIRAL (Park & Wenson)*, the design reduces the complexity of the control parameters to achieve a logarithmic spiral assembly.

To streamline the design, additional constraints were applied to the 5 available degrees of freedom:

- **Fixed Angles:** The internal structure is constrained by fixing the internal angles, reducing the variability.
- **Geometric Progression:** The side lengths follow a strict reduction ratio, creating a self-similar spiral structure.

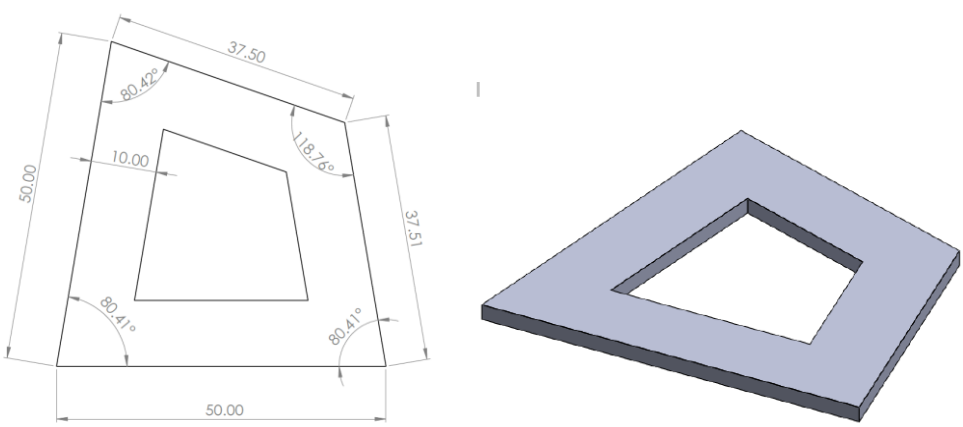
The design utilizes a recursive assembly of six quadrilaterals. The leading side of the n-th quadrilateral serves as the base for the (n+1)-th quadrilateral. The internal angles are held constant across all iterations to maintain geometric similarity:

- Angle alpha: 80.41 degrees
- beta: 80.41 degrees
- gamma: 80.42 degrees

The side lengths for the six quadrilaterals were calculated based on a fixed reduction ratio, resulting in the following dimensional series:

Quadrilateral ID	Side Length (mm)
1	50
2	37.51
3	28.14
4	21.11
5	15.85
6	11.88

(Values rounded to two decimal places for brevity, Exact values in SolidWorks files)



4. CAD Implementation and Assembly

Based on the calculated dimensions, the geometry was modeled in SolidWorks. Each quadrilateral was extruded to a thickness of 2mm. The final assembly consists of the six sequential quadrilaterals joined at their common edges. To achieve the complete curling motion achieved in the john Edmark Curling Spiral (Outer Spine) framework, a mirror copy of the primary spiral chain was assembled, completing the mechanism's symmetric profile.

The spiral assembly video is available in the documentation folder under the filename 'Spiral SolidWork visualization'.

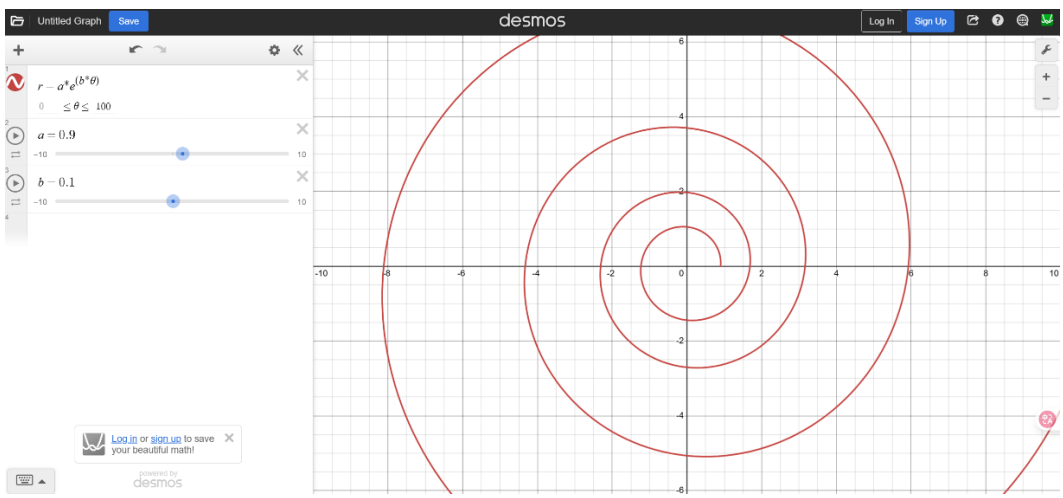
5. Spiral (Mathematical Foundation)

The mechanism's profile is derived from the logarithmic spiral. In polar coordinates (r, θ), this curve is defined by the equation:

$$r = a * e^{(b*\theta)}$$

Where:

- r (Radius): The distance from the origin (spiral center) to the curve.
- theta(Θ): The angular position of the point in radians.
- a (Scale Factor): This parameter defines the initial size of the spiral. Mathematically, it is the radius when the angle θ = 0. In the context of the mechanism, a correlate to the initial dimension of the first quadrilateral (e.g., the 50mm side).
- B (Growth Rate): This constant controls how rapidly the spiral expands. It determines the "tightness" of the wrapping. A higher value of b results in a rapid expansion, while a lower value creates a tightly wound spiral.



(Reference to [John Edmark design](#))

Week 2: Literature review and Simulation in Abaqus

6. Research paper SpiRobs summary:

Nature has already perfected the art of the soft touch, seen in the fluid movements of an octopus arm or an elephant's trunk. The SpiRobs project takes these organic blueprints and translates them into a clever geometric robotic system.

Instead of relying on sensors and hardware, SpiRobs use a smart "uncurling-to-reach" motion. By modeling a logarithmic spiral and mirroring it into a 3D shape, these robots naturally wrap themselves around objects. This action allows the robot to adapt to almost any shape automatically, shifting the complexity from the software to the actual geometry of the body.

- **Extreme Strength-to-Weight:** A prototype weighing just 38.4 g can lift a 10 kg load. That is roughly 260 times its own weight, all while being controlled by just a few simple internal cables.
- **Scalability:** The math stays the same regardless of size. The team successfully built everything from a tiny 1 cm gripper to a massive 1-meter-long arm.
- **Low-Cost & Accessible:** These aren't million-dollar machines; they are fabricated using standard desktop 3D printers and flexible TPU filament.
- **Tactile Intelligence:** Even without dedicated touch sensors, the system can detect a contact as light as a feather just by monitoring spikes in the motor's electrical current.

SpiRobs have a radius of curvature that decreases toward the tip, allowing them to wrap tightly around very small objects. SpiRobs can also perform high-speed whipping motions, allowing them to catch a flying ball or launch an object at speeds up to 10.5 m/s.

7. Research paper Elephant-Inspired grasping summary:

This paper introduces a hybrid robotic system that pairs a rigid robotic arm with a cable-driven soft manipulator to replicate the versatile grasping strategies of an elephant trunk. The rigid arm acts like the elephant's head for precise positioning, while the soft manipulator reproduces the bending and twisting motions of the trunk. By synergizing these components, the robot successfully replicated 9 out of 17 documented African elephant grasping strategies, adapting to objects of various shapes and sizes while keeping the control system relatively simple.

- The hybrid system consists of a Franka Emika Panda rigid arm and a 3D-printed soft continuum manipulator equipped with a three-fingertip gripper.
- The logarithmic spiral design allows the manipulator's curvature to initiate at the tip and dynamically propagate toward the base, directly mimicking an elephant trunk during prehension and transport.
- This model actively accounts for gravity and varying cable actuation lengths to accurately predict complex bending and twisting deformations.
- It demonstrated the ability to handle delicate items like a 10g blueberry using a pinch grip, up to heavier objects like a pineapple, showcasing versatility beyond traditional rigid industrial arms.

- The soft arm failed to securely hold a coconut because the liquid inside caused dynamic center of gravity shifts, highlighting a distinct limitation in the manipulator's torsional stiffness.

Future development aims to integrate tactile sensors, improve torsional stiffness to handle shifting loads, and potentially add a pneumatic suction cup to grasp flat or granular objects

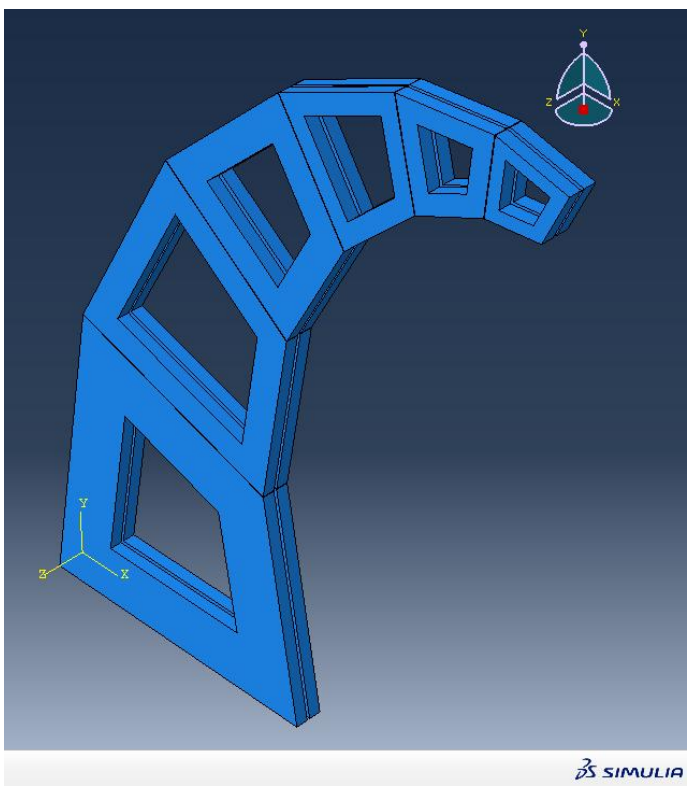
8. Modeling and Simulation Strategy in Abaqus

1. Modeling Approach and Skill Acquisition

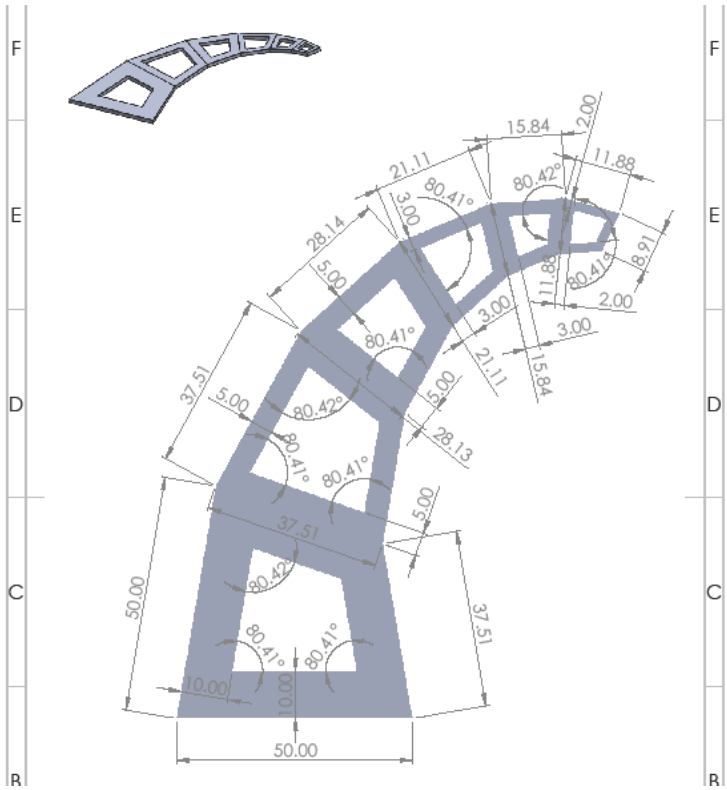
For the continuous behavior of the spiral assembly, the FEA is conducted using Abaqus CAE. While my prior work focused on 1D and 2D thermal analysis, this study required 3D structural modeling. The methodology for generating 3D geometries, the usage of datum planes and cell partitioning was developed through a review of educational resources, credits to the A-Star Education and Ahmed Elkady YouTube channels. These resources provided me the foundational steps required to transition from simple planar elements to solid modeling.

2. Geometry and Domain Discretization

Instead of modeling the plywood and tape as separate parts with contact interactions, the assembly was modeled as a single body. To define the distinct material regions, the geometry was subjected to partitioning strategies and section assignments. This allowed for different material properties to be assigned to the specific cells representing the tape and the plywood in spiral assembly.

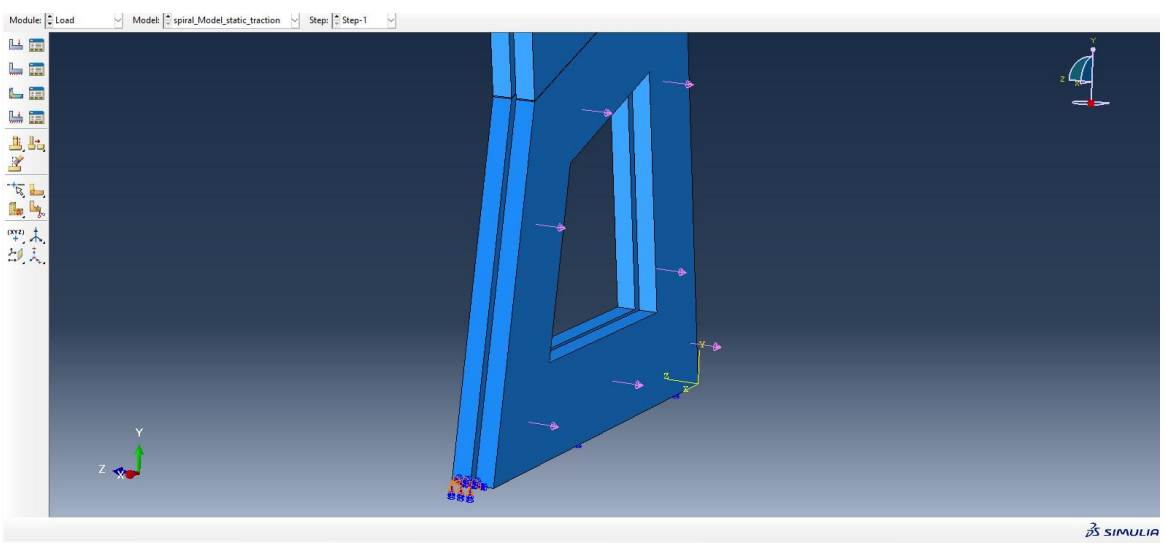


Dimensional Specifications: The specific dimensions utilized in the Abaqus model correspond to the CAD geometry previously developed in SolidWorks.



System of Units: To maintain consistency within the Abaqus environment (which is unit-neutral), SI units was adopted for this analysis.

The largest quadrilaterals serve as the regions for boundary condition application. The base of the left quadrilateral is fully constrained (fixed), whereas the opposing quadrilateral undergoes varying load conditions specifically body force, traction, and pressure across different simulation trials.



3. Material Constitutive Models

The material properties were defined as linear elastic. The parameters for density, Young's Modulus, and Poisson's ratio were derived from a preliminary literature review. While these values have not been experimentally verified for the specific samples used in this project, they serve as a representative baseline for static analysis and provide the necessary density data should dynamic analysis be required later.

Baltic Birch Plywood

Values were sourced from the study *"Elastic properties of birch plywood" (Chudoba et al., Materials & Design, Vol 25, 2012)*.

- Young's Modulus (E): 7 GPa
- Poisson's Ratio (ν): 0.13
- Density (ρ): 680 kg/m³

Masking Tape

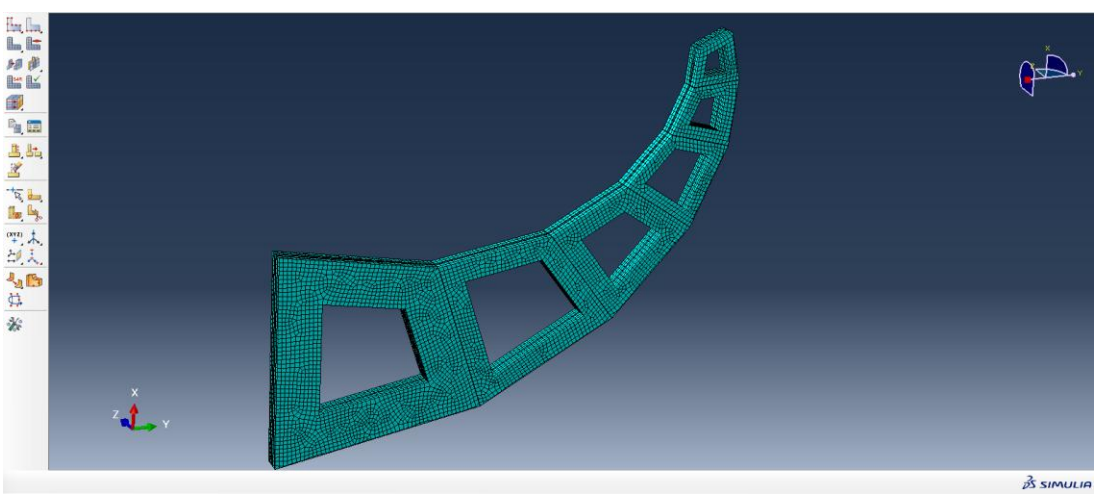
Parameters were approximated based on 3M technical data sheets (ASTM D 882 standard) and the paper *"Poisson ratio of paper" (Baum et al., TAPPI Journal)*.

- Young's Modulus (E): 0.5 GPa
- Poisson's Ratio (ν): 0.34
- Density (ρ): 880 kg/m³

4. Meshing and Element Formulation

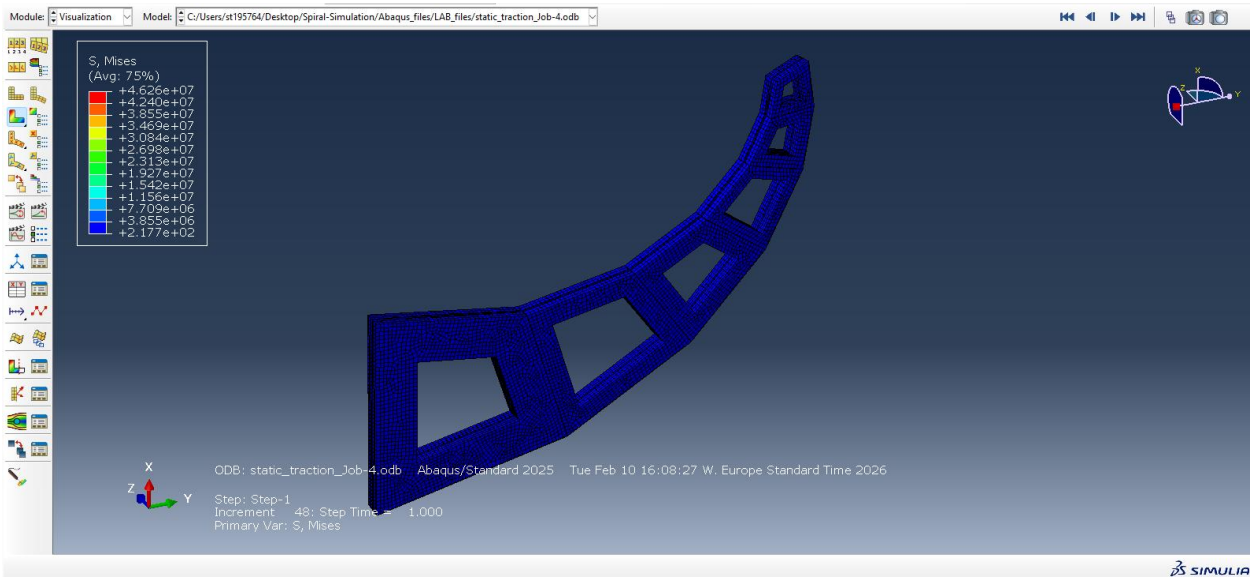
The model was discretized using 3D Stress elements. To ensure computational efficiency while maintaining accuracy, the following meshing parameters were applied:

- **Element Type:** C3D8R (8-node linear brick, reduced integration with hourglass control).
- **Element Shape:** Hexahedral (Hex).
- **Seeding:** A global seed size of 0.005 was applied, with a refined mesh size of 0.001 in critical regions.
- **Mesh Density:** These settings resulted in a total mesh count of approximately 20,000 elements.

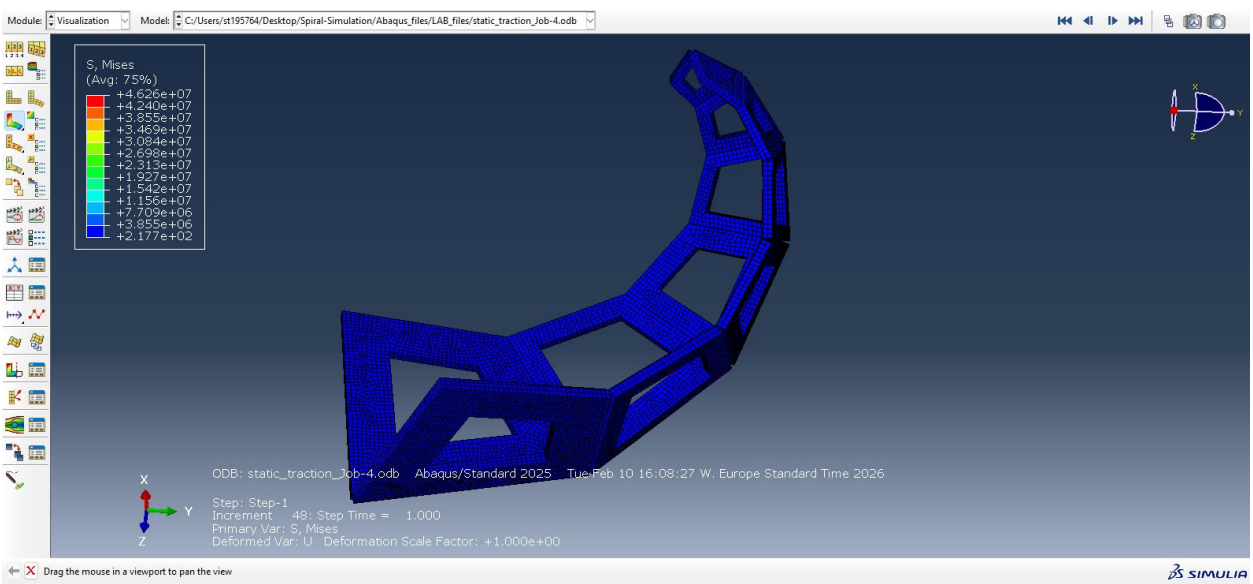


5. Result Visualization

Undeformed configuration:



Deformed logarithmic spiral:



The simulation visualization video is available in the documentation folder under the filename 'Spiral Abaqus visualization'.

Week 3: Abaqus scripting with vector mathematics

To build parametric spiral geometry within the Abaqus environment, the script cannot rely on static coordinates. Instead, it dynamically calculates the spatial position of every vertex, cut, and boundary using foundational vector mathematics and trigonometry. The following detail the core mathematical principles utilized to generate the model.

9. Coordinate Transformation

Because the vertices of the trapezoidal shapes in the spiral design are located at different local positions, it is necessary to define them with respect to a single, common global region. To accurately map every vertex to the main coordinate system, the following transformation formulas are used to calculate the exact global values:

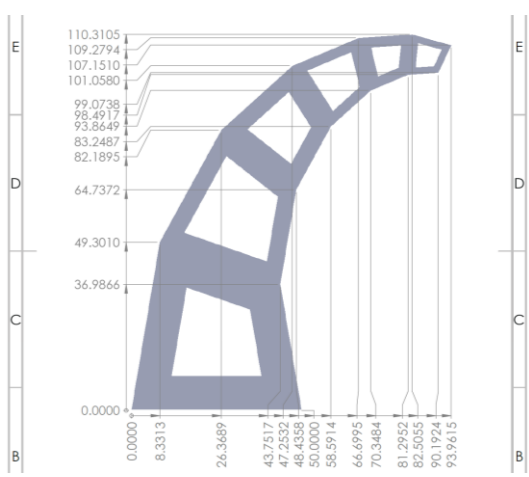
$$\begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} Xo \\ Yo \end{bmatrix}$$

In these equations, the variables represent the relationship between the local geometry of the trapezoids and the overall global space. The lower-case x and y denote the local coordinates of a specific vertex, meaning its position is measured strictly within the local grid of that particular trapezoidal shape. Conversely, the capital X and Y are the resulting global coordinates, pinpointing exactly where that vertex lies within the common, primary coordinate system.

$$X = x\cos(\theta) - y\sin(\theta) + Xo$$
$$Y = xsin(\theta) + y\cos(\theta) + Yo$$

To accurately bridge the gap between these two systems, the formulas incorporate the location of the local origin and the rotation of the grid. The terms Xo and Yo represent the global coordinates of the local origin itself—essentially marking where the local grid's starting point is placed on the main map. Finally, the angle theta defines the rotation of the entire local coordinate system, representing the angle measured from the global X-axis to the local x-axis.

A sketch of the exact location, accurate to four decimal points, of these particular vertices is given in the sketching below.



1. Trigonometric Projection and Local-to-Global Mapping

Because the spiral consists of continuous quadrilaterals segments connected end-to-end, the start of one segment depends entirely on the end of the previous one. To calculate the global coordinates of new vertices, trigonometric projections are used.

When a base edge is established, its length (L) and its angle relative to the global horizontal axis (θ_{base}) are extracted. To find the location of a new upper vertex, the local design angles (denoted as $\alpha, \beta, and\ \gamma$) are added to the base angle. The exact global coordinate of the new point is then projected using standard trigonometric functions:

$$X_new = X_base + L\,cos(\theta_base + \alpha)$$
$$Y_new = Y_base + L\,sin(\theta_base + \alpha)$$

In these equations, the local geometry of the quadrilateral is effectively translated into the global Abaqus coordinate system. This ensures that no matter how the base angles or starting widths are parametrically altered, the subsequent vertices are placed correctly in the space.

2. Linear Intersection for Vertex Determination

In many cases, a vertex is defined not by a direct length, but by the intersection of two converging lines. Such as the top and right edges of a quadrilateral closing inward. To find this exact global coordinate, the script relies on a determinant-based mathematical intersection method.

Given a starting point and a direction vector (derived from the angles calculated in the projection phase) for two separate lines, the equations calculate the exact X and Y coordinates where these trajectories cross. The system uses the determinant of the direction vectors $(C1.S2 - C2.S1)$, C & S are the cosine and sine of the line angles. If the determinant is near zero, the lines are parallel. Otherwise, it solves for the scalar distance along the first line to find the exact crossing point. This mathematical intersection is crucial for closing the geometric loops of the quadrilateral accurately without manual sketching.

3. Vector Normalization and Geometric Offsetting

To create the uniform thicknesses for the inner hollow sections and the gap cuts, the script must offset the outer perimeter lines inward by a precise mathematical distance. This is achieved through vector normalization.

For any given edge defined by the differences in its coordinates (dx & dy), the overall length of the segment is calculated using the Pythagorean theorem ($L = \sqrt{dx^2 + dy^2}$) . To find the direction that is exactly perpendicular (orthogonal) to this edge, the script calculates the unit normal vector components. By multiplying these normal vectors by the desired parametric thickness (like the wall thickness or gap width) and adding them to the original vertex coordinates, a perfectly parallel inner geometry is mathematically mapped.

4. Orthogonal Distance from a Point to a Line Segment

During the material assignment phase, the script must differentiate between regions of the geometry that are designated as "Tape" and "Plywood." Instead of manually picking these regions, the script calculates

the shortest orthogonal distance from the geometric centroid of an Abaqus cell to the dividing lines with next cell.

This mathematical check utilizes the vector dot product to project the cell's centroid onto the mathematical line defined by the divider. A mathematical clamping function (constraining the projection parameter (represented by 't' in code) strictly between 0 and 1) is used to ensure the projected point does not extend past the physical start or end of the line segment. If this computed shortest distance falls within a defined tolerance, the script mathematically identifies the cell as being part of the tape region.

10. User-Defined Parametric Variables

One of the primary advantages of this automated scripting approach is its modularity. Rather than hardcoding the geometry, the script centralizes all driving dimensions, angles, and physical properties into a single Parameters block at the beginning of the code. This allows the user to radically alter the shape, size, and physical behavior of the spiral without needing to understand or modify the complex mathematical functions detailed previously. The parameters are logically grouped into the following:

(I'm using the same variable names that I have used in the code so it is easy for you to switch between reading this document and python script understanding)

1. Global Scaling and Base Geometry

These variables dictate the foundational footprint and overall shape of the spiral.

- **scale**: A universal conversion factor (set to 0.001). Abaqus does not have a built-in unit system and relies on the user to maintain consistency. This parameter allows the user to intuitively input all dimensions in millimeters while the script automatically converts them to standard SI units (meters) for accurate physical simulation.
- **W0_mm**: The starting width of the very first trapezoidal segment. Adjusting this value scales the baseline from which all subsequent geometry is projected.
- **alpha_deg, beta_deg, and gamma_deg**: The three internal angles (defined in degrees) that dictate the shape of the repeating quadrilateral segments. Altering these angles directly changes the curvature, tightness, and trajectory of the spiral.
- **num_quads**: An integer defining the total number of segments in the design. Increasing this number forces the script to loop more times, resulting in a longer spiral with more wrapping turns.
- **thicknesses_mm**: A structured list (array) of numbers defining the internal wall thickness for each specific segment. Because it is a list, the user can create a tapered design. Making the structural walls robust at the center and progressively thinner toward the outer edges (for 6 segments I started from 10.0 mm down to 2.0 mm).
- **extrude_depth**: The total overall thickness (in meters) of the 3D part when it is pulled from a 2D sketch into a 3D solid body.

2. Cut and Hinge Geometry

To enable the spiral to fold or twist, precise Boolean cuts are made. These variables control the tolerances of those cuts, essentially defining the mechanical hinges of the structure.

- **gap_thickness_mm**: The vertical height of the slit separating the top and bottom layers of the material.
- **web_thickness_mm**: The critical dimension that leaves a thin, continuous strip of material (the "web") connecting adjacent segments. Changing this value directly impacts the rotational stiffness of the hinge, a thinner web will bend more easily but may yield under lower stress. (writing this based on my understanding of stiffness, not verified in simulation)
- **web_side**: A simple text toggle ("LEFT" or "RIGHT"). It dictates which side of the spiral perimeter remains connected as the continuous hinge, completely mirroring the folding kinematics of the final part. (This parameter was a suggestion of Gemini, although it is not useful as the quadrilateral segments are mirrored. But it can be used to study if asymmetric varying material property simulation study is performed)
- **divider_cut_width_mm & divider_cut_depth_mm**: These control the physical gaps routed between the individual trapezoidal faces. Increasing the width ensures the faces do not mathematically intersect or physically collide with each other when the spiral undergoes large rotational deformations during the simulation.

3. Material and Mesh Properties

Beyond the geometry, the script also parameterizes the finite element setup, allowing for rapid testing of different physical materials and computational fidelities.

- **Physical Properties (density, E, nu)**: The code provides variables for the density, Young's Modulus (E), and Poisson's ratio (nu) for both the "Tape" and "Plywood" sections.
- **mesh_size_mm**: Defines the global target edge length for the finite elements. A larger number results in a coarse mesh that solves quickly, ideal for draft iterations. Reducing this value generates a much denser, finer mesh that captures high-stress concentrations at the hinges, trading computational time for high-fidelity results.

11. Custom Scripting Functions

To maintain a clean, efficient, and robust script, the parametric generation process is broken down into modular custom functions with the help from Gemini. These functions abstract the complex mathematics discussed above, making the overall Abaqus generation sequence more stable and easier to iterate upon. To be honest, this was the only thing way I could figure out that the complete script could work without breaking other feature, because I faced this a lot while making the script that implementing and successfully running one particular macro would break the other part in the script.

1. intersect(pt1, theta1, pt2, theta2)

- **Purpose:** This function calculates the precise 2D spatial coordinate where two angled lines cross.
- **Input Parameters:** It takes four arguments: the global (X, Y) coordinates of a starting point on the first line (pt1), the angle of the first line (theta1), a starting point on the second line (pt2), and the angle of the second line (theta2).
- **Mechanism:** It computes the sine and cosine components for both angles and uses Cramer's rule to solve the linear system for the intersection point. It includes a safety check that returns None if the determinant is exceptionally small (indicating parallel lines).
- **Advantage in Abaqus Scripting:** Abaqus CAE requires absolute coordinates to build wireframes. By using this function, the user is freed from manually calculating the complex geometry. Changing a single parametric angle in the base variables automatically cascades through the intersect function to redefine the entire coordinate map without breaking the sketch.

2. dist_point_to_segment(px, py, P1, P2)

Gemini suggestions on defining limit to ratio were very helpful in making of this particular function

- **Purpose:** This function determines the exact distance from a free-floating point to the nearest physical location on a defined line segment.
- **Input Parameters:** It requires the (X, Y) coordinates of the target point (px, py), and the start and end coordinates of the line segment (P1, P2).
- **Mechanism:** It uses vector mathematics to project the test point onto the segment using a dot-product ratio. Crucially, it limits this ratio to a 0-to-1 range to ensure the distance is measured to the segment itself, not to an infinite continuation of the line.
- **Advantage in Abaqus Scripting:** This is used for automated topological selection. When Abaqus meshes or partitions a part, the internal cell numbering can change unpredictably. By using this geometric function to query the centroid of each cell against the design planes, the script guarantees that materials (Tape vs. Plywood) are assigned correctly based on *physical location* rather than fragile, hardcoded Abaqus array indices.

3. apply_partition(plane_feature)

- **Purpose:** A robust wrapper function to apply datum planes to slice the main 3D geometry into distinct regions.
- **Input Parameters:** It accepts a single argument: plane feature, which is an Abaqus Datum Plane object previously defined in the script.
- **Mechanism:** It instructs the Abaqus to partition all cells in the final part using the provided datum plane. The defining feature of this function is that it is wrapped in a try/except block.

- **Advantage in Abaqus Scripting:** Boolean operations and partitioning in Abaqus can fail or throw warnings if a partition plane perfectly overlaps an existing geometric boundary, or if the cell has already been split there. This function acts as a safeguard; it attempts the partition, and if Abaqus rejects it due to redundancy, it silently bypasses the error. This prevents the entire automation script from crashing over a trivial geometric overlap. (This bypass was essentially implemented because I was experiencing errors when testing more than 10 quadrilateral segments.)

4. build_face_seq(face_list)

- **Purpose:** Converts a standard Python list of geometrical faces into an Abaqus-native sequence object.
- **Input Parameters:** face_list, which is a standard Python array containing reference IDs to specific faces of the 3D model (e.g., the top and bottom surfaces targeted for boundary conditions).
- **Mechanism:** It iterates through the Python list and systematically concatenates them into an Abaqus Sequence array.
- **Advantage in Abaqus Scripting:** Abaqus is strict about data types when applying physical loads or Boundary Conditions (BCs). It will reject standard Python lists. This function acts as a necessary data-type bridge, allowing the script to logically sort the faces based on their Z-heights using Python, and then neatly package them into the precise format Abaqus requires to apply the and Displacement boundary conditions.

The Abaqus-python script is available in the 'Abaqus-files' folder under the filename 'Spiral Abaqus Script'.

Week 4: Data collection & Logarithmic spiral fitting

12. Interactive Logarithmic Spiral

To draw the spiral, the code uses the fundamental equation for a logarithmic spiral in polar coordinates:

$$r = \boldsymbol{a} * e^{(\boldsymbol{b} * \theta)}$$

- **Parameter a:** This determines the starting size and direction of the spiral.
- **Parameter b:** This controls how tightly or loosely the spiral is wound. Because it is an exponential value, even tiny changes make the spiral grow much faster.

As computer screens plot graphics using X and Y coordinates (Cartesian coordinates). To bridge this, the code includes a custom function, `calculate_spiral_coords`, which translates the polar math into standard X and Y points using trigonometry:

$$x = r\cos(\theta)$$

$$y = r\sin(\theta)$$

To ensure the spiral looks completely smooth rather than jagged, the code generates a massive sequence of 2,000 individual θ data points along the curve. Crucially, the code locks the "aspect ratio" of the graph so that circles and curves look perfectly round instead of stretched.

Instead of a static image, the script builds a user interface directly into the graph window. It creates two dedicated rectangular areas at the bottom of the screen and places interactive sliders inside them:

- **Slider A:** Allows user to drag the value of a between -1.0 and 1.0.
- **Slider B:** Allows user to drag the value of b. The code intentionally restricts this range from 0 to 0.5 because higher values would cause the exp to plot a shape too large to display cleanly.

Dynamic way to visualize how manipulating the constants of an exponential function actively changes its physical geometry in real-time.

13. Data fetching from Abaqus CAE

To analyze the deformation behavior of the simulated spiral, seven nodes data was exported from Abaqus to Excel and plotted using a Python script. It visualizes the movement of seven reference points over the duration of the simulation.

1. Starting coordinate Issue

A common challenge with extracted data from Abaqus is that the output often provides relative *displacements* over time rather than absolute spatial *coordinates*. If plotted directly, the initial shape and position of the structure would not be accurately represented in the 2D space.

To resolve this issue, the exact initial positions (at Time = 0) for all seven reference points were explicitly defined within the code. The script then iterates through the dataset, adding these hardcoded initial X and

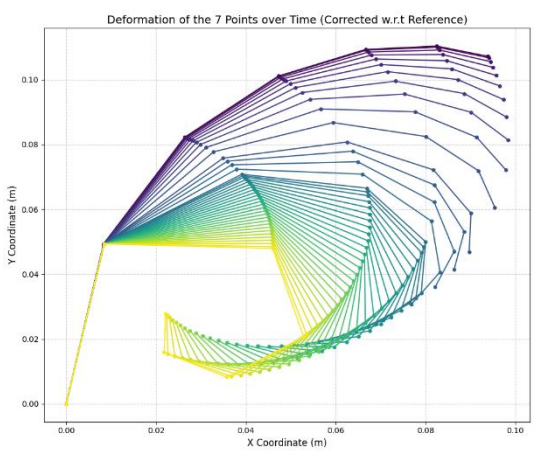
Y coordinates to the displacement values at every single time step. This correction maps the relative data back to a global coordinate system, allowing the code to capture and display the true, complete deformation of the structure accurately.

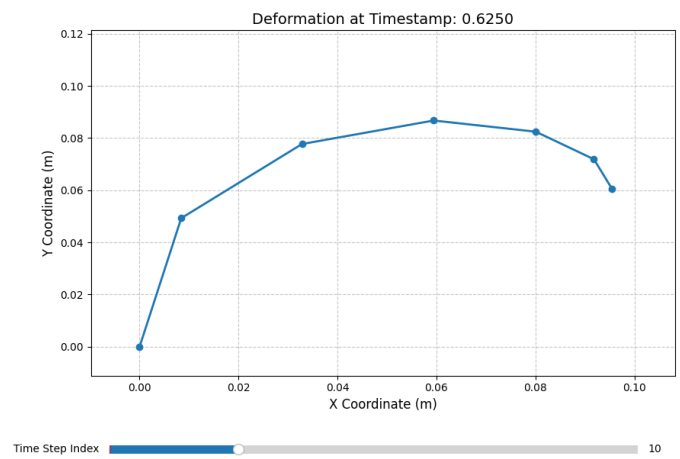
2. Visualizing the Deformation

With the absolute coordinates calculated, the script utilizes the `matplotlib` library to generate two visualizations:

- **Static Overview Plot:** This graph displays the position of the structure at every recorded time step on a single set of axes. A color gradient is applied to the lines to represent the progression of time. This provides an immediate, comprehensive overview of the total bending and the full path of movement from the beginning to the end of the simulation.

- **Interactive Dynamic Plot:** To allow for a more detailed analysis of the structural behavior, a second, interactive graph was developed. This plot features a manual slider at the bottom, enabling the user to scroll through individual time steps. As the slider moves, the graph updates to show the exact shape of the seven points at that specific moment. Making it easier to observe the dynamic deformation process.





14. Mapping of the Spiral: Code Implementation and Curve Fitting

The Python mapping script bridge the output from the Abaqus simulation with an interactive analytical model. The code workflow is divided into several key operations:

- **Data Ingestion:** The code reads the coordinate data of the quadrilateral nodes (Q1 through Q6, plus the Q6 Tip) from an Excel file. A critical first step is standardizing this data; reference coordinates are added to the raw outputs to ensure the entire mechanism is correctly positioned in the global coordinate system.
- **Mathematical Modeling:** The core of the analysis relies on fitting the nodal coordinates to the standard equation of a logarithmic spiral:

$$r = \boldsymbol{a} * e^{(\boldsymbol{b} * \theta)}$$

Where r is the radius, a is the initial scale factor, b dictates the growth rate of the spiral, and theta is the angular coordinate. The code also has a phase angle (phi) and a termination boundary (t_end) to constrain the curve's length after Quadrilateral 6 tip node.

- **Dynamic Visualization:** A slider interface is implemented using Matplotlib, allowing the user to scrub through the actuation angles (from 0° to over 152°). As the slider moves, the script fetches the corresponding nodal coordinates for that specific state and recalculates the best-fit spiral in real-time.

The quadrilaterals open across a range of 152°, but the visualization relies on only 42 discrete time steps exported from the Abaqus simulation. Because of this limited number of data points, the movement controlled by the slider appears discrete rather than perfectly continuous.

1. Automated Fitting Methodology

To map the theoretical logarithmic spiral onto the discrete nodes (the quadrilateral vertices), the code employs an optimization routine using the *scipy.optimize.minimize* function with the Nelder-Mead algorithm.

The strategy works as follows:

- **Anchoring:** The curve is strictly anchored at the Q6 Tip. This is a known, fixed boundary condition for the outer edge of the spiral.
- **Error Minimization:** The algorithm generates a trial spiral and calculates the shortest Euclidean distance from this curve to each of the remaining simulation nodes. The objective function sums these squared distances.
- **Tail Constraint:** Logarithmic spirals naturally expand outward infinitely. To prevent the algorithm from generating massive, sweeping tails that technically fit the nodes but are physically inaccurate, a soft constraint is added. This constraint penalizes the distance between the end of the generated spiral and the first node (Q1), forcing the curve to terminate near the base of the mechanism.

- **Iterative Solving:** The Nelder-Mead algorithm tests various combinations of *a, b, phi,* and *t_end* against a set of hardcoded initial guesses until it finds the combination that yields the lowest possible error.

Actuation Angle (deg)	Parameter a	Parameter b	Theta (rad)
0	0.02147	0.896	2.02
53.08	0.03191	0.601	2.16
82.91	0.02341	0.584	2.6
114.92	-0.0146	0.495	3.65
137.16	-0.0138	0.3	5.04
148.38	-0.0121	0.216	6.43
150.82	-0.0119	0.196	6.82
152.60	0.00844	-0.185	-8.28

2. Parameter Trend Analysis

As the angle between the two base quadrilaterals increases, the mechanism curls inward. This physical deformation is distinctly reflected in the optimized parameters of the logarithmic spiral.

- **Scale Factor:** The value fluctuates initially and then shifts to a negative value around 114.92°. In the context of polar coordinates, negative a value combined with the phase shift (phi) simply rotates the spirals orientation by 180 degrees. The absolute magnitude of scaling factor 'a' steadily decreases as the structure compresses.
- **Growth Rate:** There is a strict, continuous downward trend in the value of b, dropping from 0.896 to 0.196. This perfectly aligns with the physical behavior of the model as the mechanism winds tighter, the radial expansion per degree of rotation must decrease.
- **Angular Domain (Theta):** The required radians to draw the curve (theta) increases drastically from 2.02 rad to 6.82 rad. As the structure curls into a more complex, overlapping shape, a much longer section of the logarithmic spiral is required to map the full path from the tip to the base.

3. Diagnosis of Fitting Failure at high Angles (>151°)

At an actuation angle of 152.60°, the optimization function fails to find the correct geometry, producing a mirrored curve that no longer aligns with the spiral nodes.

This is an issue with unconstrained numerical optimization. The Nelder-Mead algorithm does not inherently understand the physical reality of the mechanism. It only looks for the lowest mathematical error based on the provided initial guesses.

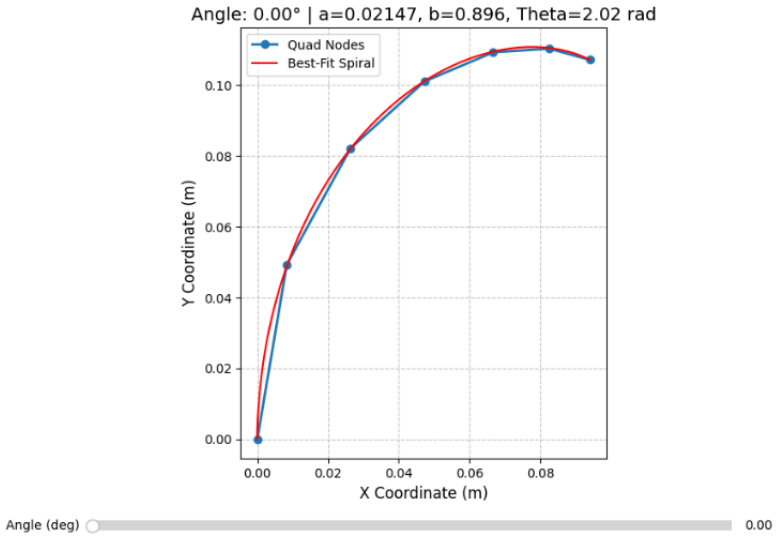
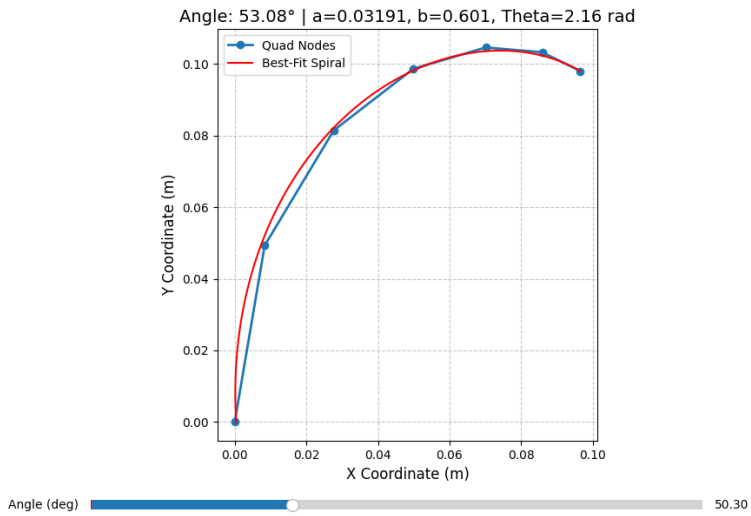
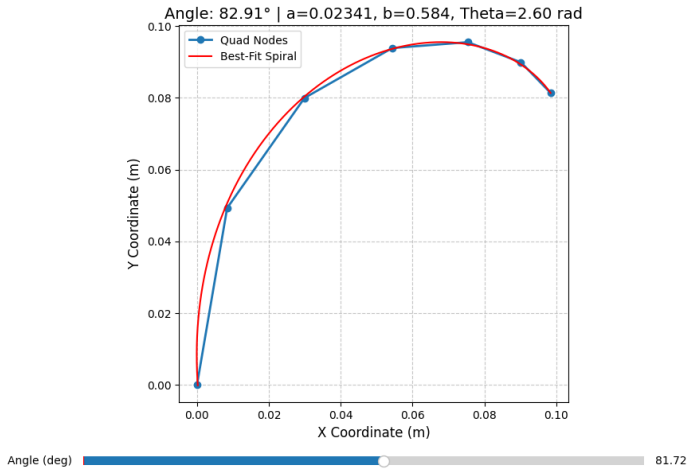
As the spiral wraps tightly (exceeding 2*pi radians), the objective function becomes highly non-convex, meaning it develops multiple valleys (local minima). At 152.60°, the default initial guesses provided in the code are no longer close enough to the true wound state. The solver gets trapped in a local minimum were flipping the parameters utilizing a negative growth rate (b = -0.185) and a negative angular domain (theta = -8.28$) yields a mathematically fit that satisfies the Q6 anchor and the Q1 tail constraint, but completely ignores the correct geometric path of the intermediate nodes.

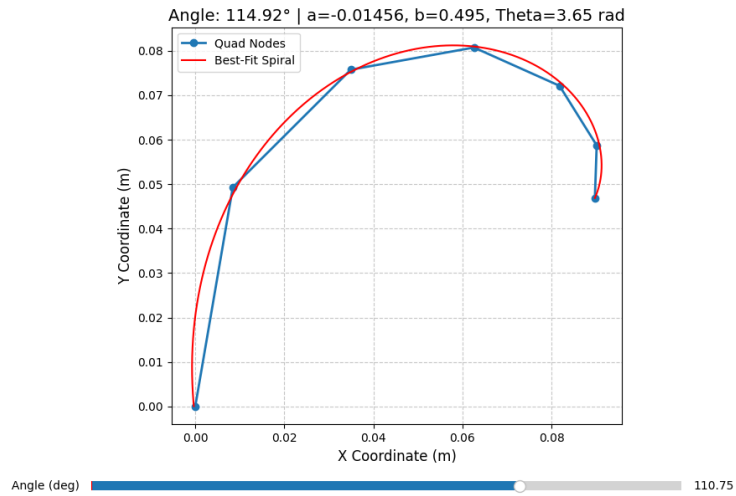
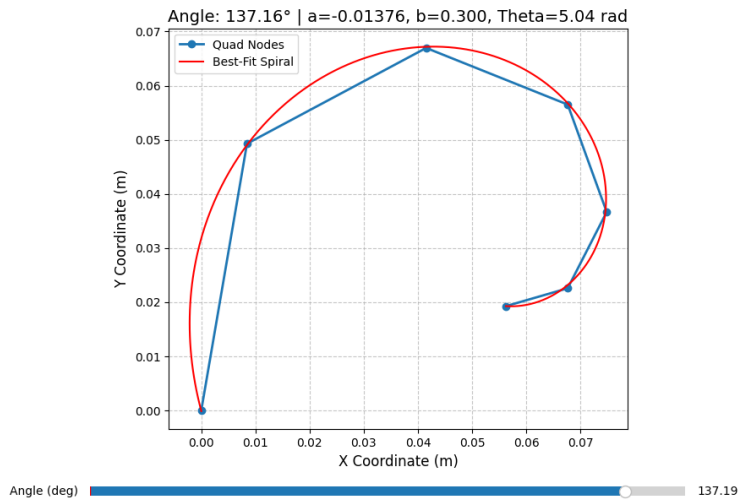
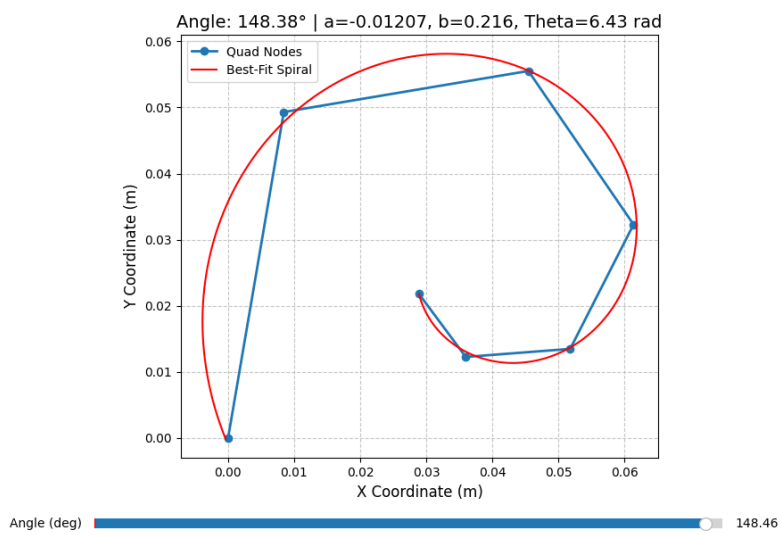
4. Visualizing the Spiral Fit Across Actuation Angles

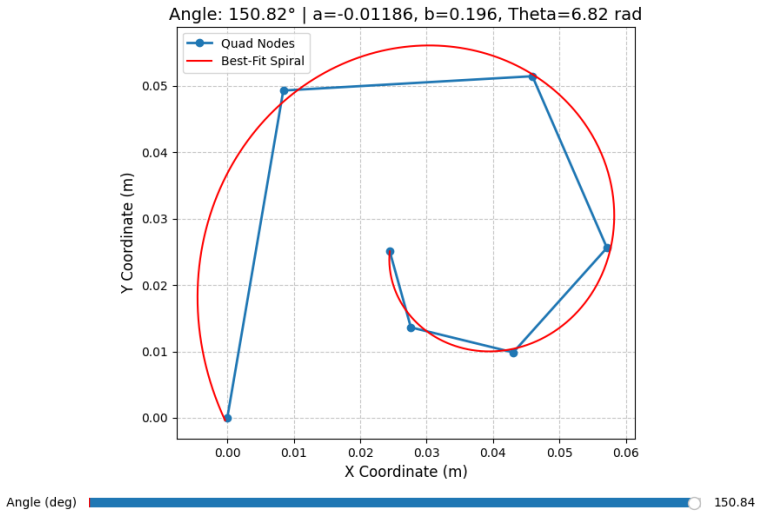
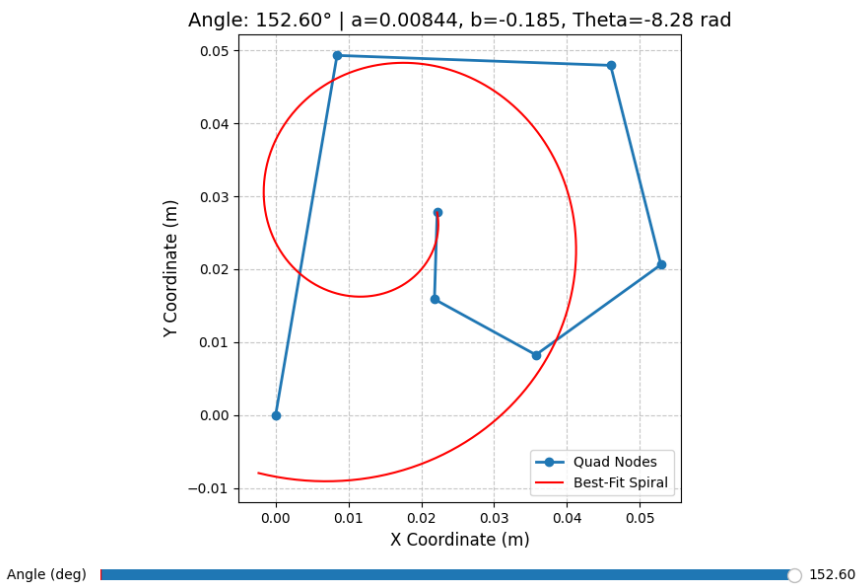
For this analysis, the actuation angle is defined as the angle between the two largest quadrilaterals at the base of the structure, which drives the entire inward curling motion.

In each of the plots below, the blue markers represent the discrete physical nodes of the mechanism extracted directly from the Abaqus simulation. The solid red line represents the continuous logarithmic spiral generated by our optimization script.

Reviewing these plots sequentially from 0.00° up to 152.60°, we can visually verify how the mathematical model adapts to the physical bending. It shows the spiral tightening as the structural angle increases, and also clearly illustrates the specific point of failure at the maximum bending angle, where the numerical best-fit solver snaps to a mirrored, physically inaccurate curve.

Actuation Angle (deg)	*Nodes and Best-fit plot*
0	
53.08	
82.91	

Actuation Angle (deg)	*Nodes and Best-fit plot*
114.92	
137.16	
148.38	

Actuation Angle (deg)	*Nodes and Best-fit plot*
150.82	
152.60	

The mapping spiral python file is located in the documentation folder.

If you like to dive deeper into the simulation, code, or plots yourself, all the relevant files are hosted online. The Python scripts, simulation files, and extended report details can be found in the [Spiral Simulation GitHub Repository].